

Industrial Roadmap for a Volatility Infrastructure Platform

*Institutional-grade implementation blueprint assuming full IBKR market-data and
order-routing access*

Prepared as an engineering roadmap for implementation by a junior developer without
disclosing any strategy logic

Version 4.0

06 April 2026

Purpose. This document specifies the target architecture, data model, mathematical framework, implementation plan, quality controls, and operational runbooks required to build a reusable volatility infrastructure. It is intentionally strategy-agnostic. It should enable a junior engineer to build the full data, pricing, surface, and risk stack while remaining blind to any downstream alpha logic.

Confidential engineering specification

Document overview

This document is written as a build manual rather than a research note. Every section is framed around implementation outcomes, interfaces, data contracts, acceptance criteria, and operational behavior. The intended reader is a junior developer who can code competently in Python but should not infer anything about the commercial intent of the system. For that reason, the document focuses on industrial primitives: market-data capture, canonical instrument master, forward reconstruction, implied-volatility inversion, surface construction, pricing, Greeks, scenario risk, persistence, orchestration, and observability.

The roadmap assumes access to Interactive Brokers Trader Workstation or IB Gateway, complete API entitlements for the relevant underlyings and option chains, and the right to store internally generated derived analytics. The system is designed so that raw market observations remain immutable, while all downstream analytics can be recomputed reproducibly from the raw layer. This separation is critical. It permits regression testing, deterministic restatement of historical analytics, and safe replacement of any one computation module without rewriting the rest of the stack.

Intended deliverables

- A production collector that discovers the tradable universe, subscribes to underlying and option market data, and stores every event with normalized timestamps.
- A canonical analytics layer that builds spot, forward, dividend/carry, implied volatility, surface parameters, model prices, and Greeks.
- A historical backfill and replay framework that can reconstruct entire days from snapshots or bars, then restate the derived analytics using the same code path as live processing.
- A scenario engine that calculates portfolio-level risk under spot, volatility, and time shocks, with configurable stress grids and clear attribution outputs.
- Operational runbooks, test plans, and handover notes so the system can be maintained by someone other than the original implementer.

What this document does not contain

- Any statement of the downstream trading strategy, position-selection logic, or portfolio-construction rule.
- Any alpha signal, timing rule, trade expression template, or strategy-specific optimization objective.
- Any recommendation on directional views, tactical execution style, or risk appetite.

Sixteen-step roadmap at a glance

Step	Workstream	Primary output	Acceptance test
1	Access, environments, and security	Running IBKR connectivity in dev and prod-like	Gateway/TWS reachable, credentials stored safely, health

Industrial Roadmap for a Volatility Infrastructure Platform

		environments	checks green
2	Instrument master	Canonical master for underlyings, expiries, strikes, multipliers	Universe reproducible from configuration and API discovery
3	Market-data ingestion	Raw quote/event store for underlyings and options	Loss-tolerant event capture with timestamp normalization
4	Persistent storage	Immutable raw layer and curated analytics layer	Backfill and live runs write identical schemas
5	Spot builder	Reliable spot or reference price series	Mid/last fallback logic works and edge cases are labeled
6	Forward and carry engine	Forward curve and implied carry/dividend curve	Parity-based forward stable across liquid strikes
7	Quote normalization and QC	Clean option quote set per maturity and strike	Illiquid or inconsistent quotes correctly filtered
8	Implied-volatility solver	Mid IV and model-consistent diagnostic outputs	Convergence, bounds, and error states logged
9	Surface engine	Interpolated surface and parameter snapshots	No obvious static arbitrage breaches after QC
10	Pricing engine	European and American pricing services	Benchmark tests pass versus reference cases
11	Greeks and risk analytics	Per-contract and portfolio Greeks	Reconciles against finite-difference checks
12	Scenario engine	Stress PnL and margin-style approximations	Worst-case loss reproducible under versioned scenarios
13	Historical reconstruction	Restated history of surfaces and risk	Replay equals same-code-path

IBKR-based volatility infrastructure blueprint

		metrics	recomputation
14	Validation framework	Automated QA, anomaly flags, audit trail	Daily QC report complete and actionable
15	Orchestration and observability	Schedulers, logs, dashboards, alerts	Failures detected quickly and restart cleanly
16	Production handover	Docs, SOPs, support model, release checklist	Junior operator can run and support system

Part I — System architecture and engineering principles

Architecture principles

Determinism

Given the same raw observations, configuration version, and code version, the system must reproduce identical derived analytics. Reproducibility is a non-negotiable property because all later QA depends on it.

Layer separation

Split the stack into five layers: connectivity, raw capture, normalized market state, derived analytics, and portfolio/risk analytics. No downstream layer may silently overwrite an upstream observation.

Idempotent processing

All batch jobs must be restartable. If a job is re-run for a given date partition, the outcome must either be byte-for-byte identical or intentionally versioned as a new analytics release.

Transparency

Every computed object must carry provenance: source timestamps, calculation timestamp, code version, config hash, and the source records used to compute it.

Operational simplicity

Prefer a simpler implementation that is inspectable and testable over an academically elegant one that is fragile in production. Reliability outranks sophistication.

Target component model

Connectivity service

A thin service responsible for establishing and maintaining IBKR sessions, authenticating to TWS or IB Gateway, and exposing a stable internal event stream. It must own reconnect logic, pacing-awareness, and heartbeat monitoring.

Universe service

A scheduled service that discovers instruments, resolves contract identifiers, validates expiries/strikes/multipliers, and materializes the canonical option universe to storage.

Collector service

A streaming or polling service that subscribes to underlyings and options, receives ticks and computations, stamps them with normalized timestamps, and writes them to the raw layer.

Analytics service

A stateless computation layer that consumes normalized market states and emits forward curves, implied volatilities, surface snapshots, model prices, and Greeks.

Portfolio/risk service

A service that merges positions with analytics snapshots, calculates sensitivities and scenario PnL, and publishes both intraday and end-of-day reports.

Control plane

The job scheduler, configuration repository, logging, metrics collection, release registry, and runbooks that allow the whole system to be operated as a product rather than a loose script collection.

Recommended deployment layout

A practical first implementation uses one repository, one shared configuration package, one relational metadata store, and one object store or partitioned file store for raw and derived datasets. The live collector should run continuously. The analytics and risk layers should run both incrementally intraday and as an end-of-day reconciliation batch. Keep the connectivity process separate from the analytics process. If the IBKR session drops, the collector can restart without invalidating the analytics layer or corrupting state.

Core naming conventions

- Use a stable composite instrument key: underlying symbol, security type, exchange, expiry, strike, option right, multiplier, currency, and the broker-specific contract identifier.
- Distinguish raw timestamps from normalized timestamps. For example: `exchange_ts`, `receipt_ts`, and `canonical_ts`.
- Version every configuration set: universe version, QC threshold version, solver version, and scenario-grid version.
- Use explicit maturity notation in years for all analytics, but store the original expiry date and the day-count convention used to convert dates into year fractions.

Part II — Mathematical framework

The volatility stack must use a consistent mathematical representation from raw quotes through risk reporting. The point is not to impose one academic model on every problem; the point is to make sure every module speaks the same language. The underlying reference should be a clean

spot or reference price, each maturity should have a reconstructed forward, the quote set should be transformed into log-moneyness, and the surface should usually be represented in total variance rather than raw volatility whenever interpolation is required.

Reference spot and market state

For each underlying, define a reference price at time t . In liquid hours, the default should be the mid-price of the live bid and ask. Outside liquid hours or when the spread is abnormally wide, the module should fall back to an explicitly labeled alternative such as last trade, official close, or a carry-forward from the most recent trusted snapshot. Never hide a fallback. The chosen reference type must be stored as a field because downstream diagnostics often need to know whether a surface was built on a live mid or a fallback reference.

$$S_t^{\text{mid}} = \frac{B_t + A_t}{2}$$

Equation 1. Mid-based reference spot

A market state snapshot is the smallest coherent state used by downstream analytics. It should contain at minimum: the underlying reference price; underlying bid, ask, last, and volume context; every eligible option bid, ask, last, size, open interest, and the broker-returned model computation fields if available; plus metadata such as trading session status, timestamps, and data-source health flags.

Forward reconstruction and carry

For an option maturity T , the system must infer a forward price $F(T)$. On equities and equity indexes, the forward embeds rates and expected carry or dividends. The preferred practical method is parity reconstruction from liquid call-put pairs because it avoids relying on a separate dividend feed in the first build. In parallel, the system may maintain a rate curve and derive an implied carry or dividend yield for diagnostic use. The parity forward should be estimated from several near-the-money strikes and combined with liquidity-aware weights rather than from a single pair.

$$F(T) \approx K + e^{rT}(C(K, T) - P(K, T))$$

Equation 2. Put-call parity forward estimate

$$F(T) = S_0 e^{(r - q)T}$$

Equation 3. Carry-based forward identity

$$\hat{F}(T) = \frac{\sum_i \omega_i F_i(T)}{\sum_i \omega_i}, \quad \omega_i = \frac{1}{\text{SpreadPct}_i + \varepsilon}$$

Equation 4. Weighted aggregation of parity forwards

$$q(T) = r(T) - \frac{1}{T} \ln\left(\frac{F(T)}{S_0}\right)$$

Equation 5. Implied carry or dividend yield from spot and forward

The junior developer should implement both a point estimate and a diagnostics bundle. The diagnostics bundle must include the list of strikes used, call and put mids, weight per strike, parity residual per strike, the chosen forward, and a quality label. This is necessary because forward errors contaminate every later quantity: moneyness, IV, surface shape, deltas, and scenario PnL.

Log-moneyness and total variance

Quotes should be mapped into log-moneyness relative to forward, not spot. This is more stable across maturities and aligns naturally with total-variance parameterizations. For each accepted option quote, compute $k = \ln(K/F(T))$. Also compute total variance $w = \sigma^2 T$. Interpolate and smooth in total variance space whenever possible. A surface represented as total variance is usually easier to compare across maturities and easier to constrain for basic static arbitrage conditions.

$$k = \ln\left(\frac{K}{F(T)}\right)$$

Equation 6. Log-moneyness

$$w(k, T) = \sigma_{\text{imp}}(k, T)^2 T$$

Equation 7. Total variance

European pricing identities

European options should be priced using a forward-consistent form of Black-Scholes/Black-76. The implementation should expose both direct price functions and inverse functions that solve for implied volatility from a market price. The solver must be robust to deep in-the-money, deep out-of-the-money, short-dated, and stale-quote edge cases. It must expose convergence status, number of iterations, lower and upper bounds used, and the final residual.

$$d_1 = \frac{\ln(S_0/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}$$

Equation 8. Black-Scholes d_1

$$d_2 = d_1 - \sigma\sqrt{T}$$

Equation 9. Black-Scholes d_2

$$C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2)$$

Equation 10. European call price

$$P = K e^{-rT} N(-d_2) - S_0 e^{-qT} N(-d_1)$$

Equation 11. European put price

American pricing identities

For single-name equity options that can be exercised early, the first production version should expose a lattice-based pricer and may optionally add a closed-form approximation such as Bjerksund-Stensland for speed. The implementation target is not novelty but stable, testable behavior. The developer must document which pricer is used where, what carry assumptions are passed in, and how the early-exercise check is implemented. The pricer must also be benchmarked against degenerate cases where the American price should converge to the European price.

$$V_{n,j} = \max(\Phi(S_{n,j}), e^{-r\Delta t} [pV_{n+1,j+1} + (1-p)V_{n+1,j}])$$

Equation 12. Backward induction for an American-option tree

Greeks and risk identities

All first-order and second-order sensitivities should be computed in a unified unit system. Delta and gamma should be defined with respect to the underlying reference price; vega should be defined per absolute volatility point or per unit volatility depending on the code convention, but the convention must be explicit and consistent. For portfolio and desk reporting, also compute monetized Greeks such as dollar gamma and dollar vega.

$$\Delta = \frac{\partial V}{\partial S}$$

Equation 13. Delta

$$\Gamma = \frac{\partial^2 V}{\partial S^2}$$

Equation 14. Gamma

$$\mathcal{V} = \frac{\partial V}{\partial \sigma}$$

Equation 15. Vega

$$\Theta = \frac{\partial V}{\partial t}$$

Equation 16. Theta

$$\text{DollarGamma} = \Gamma S^2 \times \text{Multiplier}$$

Equation 17. Dollar gamma

$$\text{DollarVega} = \mathcal{V} \times \text{Multiplier}$$

Equation 18. Dollar vega

$$\Delta V \approx \Delta dS + \frac{1}{2} \Gamma (dS)^2 + \mathcal{V} d\sigma + \Theta dt$$

Equation 19. Local PnL approximation from Greeks

Surface parameterization and no-arbitrage diagnostics

The volatility surface module should support at least one parameterized representation and one nonparametric fallback. A practical parameterized choice is SVI by maturity slice. The fallback can be a spline or monotone interpolation in total variance space after stringent QC. The goal is not to force one model everywhere, but to produce a stable, interrogable surface. The system should test at minimum for obvious calendar inconsistencies and for gross cross-strike pathologies. Advanced no-arbitrage enforcement can be added in later releases, but basic diagnostics must be present from day one.

$$w(k) = a + b \left(\rho(k - m) + \sqrt{(k - m)^2 + \sigma^2} \right)$$

Equation 20. SVI slice parameterization

$$\frac{\partial w(k, T)}{\partial T} \geq 0$$

Equation 21. Basic calendar monotonicity condition

$$w(T) = \lambda w(T_1) + (1 - \lambda) w(T_2)$$

Equation 22. Variance interpolation across maturities

Index or basket variance identity

The infrastructure may need to compute basket-level or index-level variance identities for generic correlation diagnostics. The system should therefore include a generic implementation of

the weighted variance identity. This is not strategy logic; it is a reusable risk and diagnostics primitive. The module must accept a vector of weights, constituent volatilities, and optional pairwise correlations or a simplifying average-correlation assumption, then return the implied basket variance and residual metrics.

$$\sigma_I^2 \approx \sum_i w_i^2 \sigma_i^2 + \sum_{i \neq j} w_i w_j \rho_{ij} \sigma_i \sigma_j$$

Equation 23. Generic basket variance identity

Part III — Sixteen-step implementation roadmap

This part is the core delivery. Each step contains a practical objective, a task list, expected outputs, acceptance tests, and implementation notes. The steps are ordered deliberately. A junior developer should not skip ahead. Later stages depend on decisions made and artifacts created earlier. If a later step appears blocked, the correct remedy is usually to strengthen the preceding step rather than to patch around the problem.

Step 1 — Access, environments, and security

Objective

Create a safe, repeatable foundation for all later work. The junior developer must set up a development environment, a production-like environment, and a configuration/secret management pattern before writing any analytics code. A common failure mode in early trading infrastructure is to mix credentials, hand-edited notebooks, and ad hoc scripts. This step eliminates that risk by imposing structure from the beginning.

Detailed tasks

(a) Install Python and package management tooling; pin versions; produce a reproducible lock file. (b) Stand up TWS or, preferably for unattended operation, IB Gateway in an isolated user account. (c) Configure an application-level client ID convention so multiple services do not collide. (d) Build secret loading via environment variables or a secret manager, never via hard-coded strings. (e) Create configuration files for exchanges, instruments, calendars, and QC thresholds. (f) Add basic health checks: API reachable, login valid, market-data entitlement present, and clock synchronization within tolerance. (g) Decide and document where logs and artifacts will be written in each environment.

Outputs

A reproducible Python environment, a running IBKR session reachable from code, a configuration package under version control, and a bootstrap script that proves end-to-end connectivity without placing orders. The bootstrap script should print the session state, current time, contract resolution result for one underlying, and one market-data retrieval example.

Acceptance criteria

A new machine can be provisioned from documentation; the bootstrap script succeeds; secrets are not stored in the repository; all environment-specific values are loaded through documented configuration; and a simple connectivity job can be run from the scheduler without manual intervention.

Junior implementation notes

Start with the smallest useful smoke test. Resolve one contract, request one quote, and write one JSON line to disk. Once that succeeds reliably, only then add complexity. Keep a markdown runbook called `environment.md` updated as you go. Every manual step performed during setup must be documented immediately, otherwise the environment will become impossible to reproduce.

Step 2 — Instrument master and universe discovery

Objective

Build the canonical instrument master that every other service will use. The instrument master is the single source of truth for underlyings, option contracts, multipliers, currencies, exchanges, and expiries. Without this layer, analytics become brittle because the same real-world contract may be represented by inconsistent ad hoc identifiers across scripts.

Detailed tasks

(a) Define the canonical schema for underlying instruments and option instruments. (b) Implement contract-resolution helpers that map human-readable requests into broker contract identifiers. (c) Use the option-chain discovery APIs to obtain expiries, strikes, trading class, and multiplier information. (d) Normalize expiries into a consistent date format and strikes into numeric values. (e) Persist both the raw broker response and the normalized canonical representation. (f) Add data-quality checks for duplicate contracts, impossible multipliers, or missing fields. (g) Version the discovered universe by date and configuration so the same trading day can always be reconstructed.

Outputs

A canonical instrument master table plus helper methods: `get_underlying(symbol)`, `get_option_chain(symbol, date)`, `resolve_contract(key)`, and `load_active_universe(session_date)`. The table should support filtering by exchange, product type, maturity window, and listing status.

Acceptance criteria

Given a configured underlying, the system can reproduce the same active option universe on repeated runs; duplicates are removed deterministically; multiplier and currency are always populated; and any unresolved contract is surfaced as an explicit exception with diagnostics rather than silently skipped.

Junior implementation notes

Treat the broker's contract identifier as an external foreign key, not as your only identifier. Your canonical key must remain meaningful even if the broker session changes. Store raw contract payloads as evidence. Future debugging often depends on seeing exactly what the broker returned at discovery time.

Step 3 — Market-data ingestion layer

Objective

Capture raw underlying and option observations in a way that is robust to disconnects, pacing issues, and intermittent field availability. The raw layer is the evidentiary record of what the system saw. It must be append-only and loss-aware. If data is missing, that fact must be recorded rather than papered over.

Detailed tasks

(a) Decide which subscriptions are persistent streaming subscriptions and which are snapshots. (b) Create a collector process for underlying quotes and another for options, or a unified collector with explicit partitioning. (c) Normalize incoming ticks into a common event structure with instrument key, field name, value, source timestamp if available, receipt timestamp, and collector timestamp. (d) Persist every event to the raw layer with a session identifier. (e) Add reconnect logic with backoff and heartbeat monitoring. (f) Detect market-data pacing or entitlement failures and log them as structured events. (g) Build daily collector summaries: event counts, missing intervals, reconnect count, and coverage ratios.

Outputs

An append-only raw event store, a collector service process, health metrics, and a session summary report. The collector should emit enough metadata to reconstruct whether a downstream analytic used a fresh or stale observation.

Acceptance criteria

The collector can run for an entire session without manual supervision; disconnects produce warnings and controlled recovery; a synthetic kill-and-restart test does not corrupt the raw store; and at least one day of data can be replayed from disk without reaching back to the broker.

Junior implementation notes

Never compute analytics inside the collector callback itself. The callback should only normalize, stamp, and persist. Heavy logic inside the callback is the fastest path to dropped events and undebuggable behavior.

Step 4 — Persistent storage and data model

Objective

Define durable storage for immutable raw data and curated analytics. The storage design must support both live incremental writes and historical backfills. The same schemas should be used in both cases so that replay and live computation share code rather than diverge.

Detailed tasks

(a) Choose a metadata store for configuration, jobs, and reference entities. (b) Choose a columnar partitioned store for large raw and derived datasets. (c) Design partitioning by trade date, underlying, and data layer. (d) Create schema definitions for raw events, normalized market-state snapshots, forwards, implied-vol points, surface parameters, model prices, Greeks, scenarios, positions, and QC results. (e) Enforce schema evolution rules and backfill compatibility. (f) Decide retention policy for raw tick/event data, normalized snapshots, and derived analytics. (g) Add write-ahead validation so malformed records are rejected early with explicit logs.

Outputs

Versioned schemas, migration scripts, partitioned datasets, and a documented data lineage from raw events through curated analytics. The system should be able to answer the question 'which raw records produced this surface snapshot?' within one query or one notebook cell.

Acceptance criteria

All required tables exist; partitioning supports efficient daily queries; replay and live writes land in identical schemas; and deleting/recomputing one derived partition does not require rewriting the raw layer.

Junior implementation notes

Use simple, explicit schemas. Avoid storing nested structures unless they materially reduce complexity. A junior developer should favor readability and debuggability over compact cleverness. Do not mix timestamps in different time zones in the same field.

Step 5 — Spot builder and market-state snapshots

Objective

Transform raw ticks into coherent market-state snapshots for each underlying and each option chain. Snapshots are the deterministic inputs to all downstream analytics. They must be time-aligned, quality-labeled, and reproducible from raw events.

Detailed tasks

(a) Define the snapshot frequency or trigger, for example every N seconds, every material update, or on demand. (b) For each snapshot, compute reference spot using mid-price when reliable and documented fallbacks when not. (c) Store bid, ask, last, spread percentage, and the chosen reference-type flag. (d) Join options into the same snapshot with the most recent eligible quote not older than a configurable age threshold. (e) Add market-state flags such as open/closed, stale

underlying, stale option, or fallback spot. (f) Build snapshot completeness metrics per underlying and maturity.

Outputs

A normalized market-state dataset keyed by timestamp, underlying, and option contract, including all fields required by the pricing and surface layers.

Acceptance criteria

Given the same raw events and the same snapshot parameters, repeated runs produce identical market-state rows. Stale-option logic is visible in flags. Spot fallbacks are labeled, not hidden. Spot diagnostics are queryable by day and time.

Junior implementation notes

Keep the snapshot builder pure: raw events in, snapshots out. Do not call external services from inside it. That purity makes it easy to replay past sessions and to unit-test the builder with synthetic event streams.

$$S_t^{\text{mid}} = \frac{B_t + A_t}{2}$$

Step 6 — Forward and implied carry engine

Objective

Compute a robust forward for each maturity and derive an implied carry or dividend diagnostic curve. This engine is foundational because it determines the moneyness coordinate used by the implied-volatility and surface modules.

Detailed tasks

(a) For each maturity, identify eligible call-put pairs near the money. (b) Compute call and put mids, then parity forward per strike. (c) Weight forward candidates by liquidity and quote quality. (d) Remove outliers using robust statistics such as median absolute deviation or parity residual thresholds. (e) Fit or smooth the forward term structure across maturities. (f) If a rate curve is available, derive implied carry/dividend yield and compare it with expectations. (g) Persist both the chosen forward and all diagnostics used to choose it.

Outputs

A forward curve dataset by underlying and maturity plus a diagnostics dataset listing candidate strikes, weights, residuals, and quality labels.

Acceptance criteria

The forward is stable across small perturbations in the eligible strike set; outlier pairs do not dominate the estimate; diagnostics can explain why a maturity was flagged poor quality; and the forward term structure behaves sensibly across adjacent maturities unless explicitly flagged as unreliable.

Junior implementation notes

Debug the forward engine first on a handful of liquid maturities before scaling to the whole chain. When a maturity fails, inspect the raw quotes rather than trying to tune the smoother blindly. In practice, most forward errors originate in quote quality, not in the formula.

$$F(T) \approx K + e^{rT}(C(K, T) - P(K, T))$$

$$q(T) = r(T) - \frac{1}{T} \ln\left(\frac{F(T)}{S_0}\right)$$

Step 7 — Quote normalization and quality control

Objective

Establish a defensible process for deciding which option quotes may enter the solver and surface layers. The point is not to maximize quote count; the point is to maximize the number of quotes that are economically meaningful and consistent with a tradable state.

Detailed tasks

(a) Compute standard quote-quality features: spread percentage, bid positivity, quote age, open interest, volume, and monotonicity hints. (b) Mark quotes as usable, caution, or reject. (c) Detect crossed or locked markets, impossible prices relative to intrinsic value, and stale last prices. (d) Apply robust outlier statistics to parity residuals or preliminary implied volatilities. (e) Store the reason code for each rejected quote. (f) Keep both the full raw snapshot and the filtered snapshot so QC decisions are auditable.

Outputs

A filtered quote set ready for inversion plus a QC table explaining every rejection or downgrade.

Acceptance criteria

The same quote is either consistently accepted or rejected under a fixed threshold version. QC reason codes are exhaustive and mutually understandable. The filtered chain retains enough points for a stable surface while removing obvious garbage.

Junior implementation notes

Do not implement QC as a monolithic if-statement. Break it into named checks and log each check separately. This makes threshold tuning and postmortem analysis far easier.

Step 8 — Implied-volatility inversion engine

Objective

Convert filtered option prices into implied volatilities using robust numerical methods. The engine must support diagnostics, error handling, and deterministic behavior. Every solved IV should be accompanied by enough metadata to understand how reliable it is.

Detailed tasks

(a) Implement European inversion using a bracketed root solver for products where Black-style pricing is appropriate. (b) For American options, either invert through the chosen American pricer or compute a proxy IV under a documented convention. (c) Use intrinsic-value and no-arbitrage bounds to detect unsolvable inputs. (d) Record convergence status, iteration count, final residual, lower and upper bracket, and the pricing model used. (e) Expose fallback behavior for short-dated or near-intrinsic cases. (f) Build a batch interface that can solve an entire chain efficiently.

Outputs

A table of implied-vol points with coordinates in strike, moneyness, maturity, delta if available, and extensive diagnostics.

Acceptance criteria

Most liquid quotes converge cleanly; pathological cases are explicitly labeled; the solver passes reference examples; and finite perturbations in input price produce plausible IV changes rather than numerical explosions.

Junior implementation notes

Write the scalar solver first, then the vectorized batch wrapper. Keep the scalar path readable and test it thoroughly. Most production bugs are easiest to find in the scalar implementation before optimization layers are added.

$$C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2)$$

$$P = K e^{-rT} N(-d_2) - S_0 e^{-qT} N(-d_1)$$

Step 9 — Surface engine and parameter storage

Objective

Build volatility surfaces by maturity and across maturities, while keeping the raw solved points and the final fitted representation separate. The surface module should produce both a machine-friendly fitted form and a human-debuggable diagnostics package.

Detailed tasks

(a) Group implied-vol points by maturity. (b) Transform points into log-moneyness and total variance. (c) Fit the chosen parameterization slice by slice or apply the chosen nonparametric smoothing method. (d) Interpolate across maturities in variance space. (e) Compute goodness-of-fit metrics and no-arbitrage diagnostics. (f) Save both fitted parameters and reconstructed grid values. (g) Store rejected points and fit warnings so operators can see whether a smooth surface was built on very sparse input.

Outputs

Surface parameter tables, a regularized surface grid, fit diagnostics, and quality flags by maturity and by underlying.

Acceptance criteria

The fitted surface reproduces accepted market points within tolerance, diagnostics reveal sparse or poor-quality fits, and repeated runs under the same inputs return the same parameters. At least one plotting utility can visualize the raw points versus fitted slices for operator review.

Junior implementation notes

Never discard the raw solved points after the fit. Operators must be able to compare the fitted surface with the exact points that entered the calibration. This is essential for debugging suspicious Greeks or scenario outputs later.

$$w(k) = a + b \left(\rho(k - m) + \sqrt{(k - m)^2 + \sigma^2} \right)$$

Step 10 — Pricing engine

Objective

Provide reusable pricing services for both European and American options. The pricing engine is the only module allowed to translate a state vector (spot, forward, maturity, volatility, carry parameters) into price and Greeks. Centralizing this logic prevents drift across notebooks and services.

Detailed tasks

(a) Implement a European pricer consistent with the inversion engine. (b) Implement an American pricer suitable for single-name use. (c) Expose first-order and second-order Greeks, either analytically when available or numerically with documented finite-difference settings. (d) Create a clean Python API that accepts a typed input object and returns a typed result object. (e) Add benchmark tests and performance tests. (f) Document unit conventions rigorously.

Outputs

A pricing package with scalar and vectorized interfaces, complete docstrings, examples, and benchmark fixtures.

Acceptance criteria

Reference cases match expected values; the European and American engines agree in regimes where they should; unit tests cover sign conventions and limiting cases; and the API is stable enough to be imported by analytics, risk, and scenario modules without special casing.

Junior implementation notes

Avoid embedding pricing logic directly in dataframes or notebooks. Keep the pricer a clean library with tests. The clearer this boundary is, the safer later refactors become.

Step 11 — Greeks and per-position risk analytics

Objective

Turn the pricing layer into a per-contract and per-position risk service. The output of this step should be the canonical risk snapshot used everywhere else in the system, including scenario analysis and operational dashboards.

Detailed tasks

(a) Define the sensitivity set required at instrument level and at portfolio level. (b) Pull in the latest positions or hypothetical positions from the source of record. (c) Join positions to analytics snapshots and compute per-line price, Greeks, and monetized sensitivities. (d) Aggregate by instrument, maturity, underlying, and any desk-defined grouping keys. (e) Compute reconciliation checks against broker-returned Greeks if available. (f) Publish both line-level and aggregate outputs.

Outputs

Position-level risk tables, aggregated risk tables, and reconciliation reports.

Acceptance criteria

The same positions on the same analytics snapshot always produce the same aggregate risk; dollar gamma and dollar vega conventions are documented and stable; and reconciliation discrepancies beyond threshold are surfaced automatically.

Junior implementation notes

Store both contract-level and aggregate outputs. Aggregates are convenient, but debugging always starts at the line level. If a total Greek looks wrong and the line-level breakdown is missing, the system becomes opaque immediately.

$$\text{DollarGamma} = \Gamma S^2 \times \text{Multiplier}$$

Step 12 — Scenario engine and margin-style diagnostics

Objective

Build a scenario framework that approximates worst-case losses under configured spot, volatility, and time shocks. The purpose is generic risk control, capacity planning, and margin-style diagnostics. It should not encode any strategy logic.

Detailed tasks

(a) Define versioned scenario grids: spot up/down moves, volatility shifts, curve rotations if required, and time roll-down. (b) Reprice the full portfolio under every scenario. (c) Attribute PnL by line, underlying, and scenario family. (d) Compute worst-case loss, top contributors, and pathwise diagnostics. (e) Support both full repricing and local Greeks-based approximations for speed, while keeping the full repricing as the reference. (f) Persist the exact scenario set used for every report.

Outputs

Scenario result tables, worst-case summaries, and margin-style approximation reports.

Acceptance criteria

A report can be regenerated exactly given the positions, analytics snapshot, and scenario version. Worst-case contributors are explainable. Repricing and Greeks approximations agree within documented limits for small shocks.

Junior implementation notes

Version the scenario grid. Do not leave it as a mutable notebook cell. The scenario definition is part of the data lineage and must be queryable alongside the output.

$$\Delta V \approx \Delta dS + \frac{1}{2} \Gamma (dS)^2 + \nu d\sigma + \Theta dt$$

Step 13 — Historical reconstruction and replay

Objective

Enable the system to reconstruct historical analytics from stored raw data or historical snapshots using the same code path as live analytics. This step turns the infrastructure from a live monitor into a real research and audit platform.

Detailed tasks

(a) Define how a historical day is replayed: from raw events into snapshots, from snapshots into forwards and surfaces, and from surfaces into risk. (b) Ensure all derived jobs can run in batch mode over a date range. (c) Detect missing raw partitions and create partial-data flags. (d) Store restated outputs in versioned partitions so newer code versions do not silently overwrite older historical analytics. (e) Compare replay outputs with live outputs for overlapping periods.

Outputs

Replay scripts, backfill jobs, replay QA reports, and a historical archive of derived analytics.

Acceptance criteria

At least one historical month can be reconstructed end to end. Replay and live outputs align on overlapping dates when using the same code version. Missing data is flagged rather than masked by interpolation without notice.

Junior implementation notes

The replay code path should call the same libraries as live processing. Resist the temptation to fork a separate 'historical only' implementation. Dual code paths always drift and become inconsistent.

Step 14 — Validation framework and anomaly detection

Objective

Create the automated controls that determine whether each daily analytics run is trustworthy. Validation must be treated as a product, not as a last-minute dashboard. The framework should produce actionable flags instead of vague 'data looked weird' statements.

Detailed tasks

(a) Define validation checks for coverage, stale-data rates, forward stability, solver convergence, surface smoothness, no-arbitrage diagnostics, and reconciliation deltas. (b) Build daily reports summarizing pass/warn/fail outcomes. (c) Add anomaly detection against rolling baselines for key metrics such as quote counts, forward residuals, fit errors, and scenario losses. (d) Write every failed record to a triage table with enough context to investigate. (e) Define escalation thresholds and who gets notified.

Outputs

QC reports, anomaly tables, a validation dashboard, and triage views for failed instruments or maturities.

Acceptance criteria

A daily operator can identify the failing underlyings or maturities within minutes. Every failed validation has a reason code and supporting context. Historical trends of QC metrics are visible for regression monitoring.

Junior implementation notes

A good validation framework is specific. Avoid generic red banners. Operators need to know which maturity failed, which quote count collapsed, which solver residual blew out, and where to look next.

Step 15 — Orchestration, logging, and observability

Objective

Turn the build into an operable system with schedules, retries, metrics, and alerting. A correct algorithm that cannot be monitored is not production-ready.

Detailed tasks

(a) Define jobs for universe refresh, live collection, incremental analytics, end-of-day reconciliation, replay, and QC. (b) Add structured logging with correlation IDs linking collector sessions to analytics jobs. (c) Expose metrics such as event rates, stale ratios, forward failures, solver failure counts, and scenario run times. (d) Create alerts for collector death, missing partitions, elevated failure rates, and QC fails. (e) Implement restart procedures that do not duplicate or corrupt records. (f) Build simple dashboards that show system health and latest analytics coverage.

Outputs

Scheduled jobs, logs, dashboards, and alert routes.

Acceptance criteria

A simulated failure of the collector or analytics service is detected within a documented interval. Restarting a failed job does not silently duplicate outputs. Operators can identify the last healthy run and current backlog instantly.

Junior implementation notes

Prefer fewer well-labeled metrics over many opaque ones. An operator should be able to answer: Is data flowing? Are surfaces building? Are QC checks passing? Are scenario reports current? Start there and expand only if needed.

Step 16 — Production hardening, documentation, and handover

Objective

Finish the infrastructure as a maintainable product. The last step is not code; it is ensuring that another person can operate, support, and extend the platform without reverse-engineering it.

Detailed tasks

(a) Freeze interface contracts and publish them in documentation. (b) Create onboarding notes, runbooks, deployment instructions, and a release checklist. (c) Record known limitations and future enhancements. (d) Add ownership and support expectations. (e) Define change-management rules for schema changes, threshold changes, and model changes. (f) Conduct a handover walkthrough where the junior engineer demonstrates operation from environment setup through QC interpretation.

Outputs

A maintained documentation set, a release checklist, SOPs, and a support model.

Acceptance criteria

A new engineer can set up the environment, run a connectivity smoke test, trigger a replay, read the QC report, and explain where to investigate a failed surface build without support from the original author.

Junior implementation notes

Documentation is not an afterthought. Treat it like code. Every module should have a README, every public function should have docstrings, and every recurring operational procedure should exist as a runbook with concrete commands and screenshots where useful.

Part IV — Detailed implementation guides

A. Python repository structure

A junior developer should not begin with a flat directory of scripts. The repository should be organized around stable packages and clear ownership boundaries. One recommended structure is shown below. The exact names may vary, but the separation of concerns should remain.

- repo_root/
 - configs/ # environment files, thresholds, calendars, scenario grids
 - docs/ # runbooks, module READMEs, architecture diagrams
 - src/
 - connectivity/ # IBKR session management, reconnect logic, heartbeats
 - universe/ # contract resolution, option-chain discovery, master tables
 - collectors/ # raw event capture, polling/stream adapters
 - snapshots/ # normalized market-state builders
 - forwards/ # parity forward engine, carry diagnostics
 - iv/ # pricing inversion and diagnostics
 - surfaces/ # fitters, interpolation, no-arb checks
 - pricing/ # European and American pricers
 - risk/ # Greeks, aggregation, scenarios
 - storage/ # schema definitions, read/write adapters
 - orchestration/ # job entry points, scheduler wrappers
 - qc/ # validation checks and anomaly detection
 - utils/ # dates, calendars, math helpers, logging helpers
 - tests/ # unit tests, integration tests, regression fixtures

Industrial Roadmap for a Volatility Infrastructure Platform

- notebooks/ # strictly exploratory; never source-of-truth code
- scripts/ # operational wrappers for CLI entry points

Coding standards

- Every module must expose typed functions or dataclasses for inputs and outputs.
- Do not pass loose pandas rows between modules as implicit contracts. Define explicit objects.
- Separate pure calculations from I/O. Pure functions are easier to test and to replay historically.
- Any function with hidden defaults affecting economics must document those defaults in the docstring.
- Every public module should contain one worked example that a junior engineer can run.

B. IBKR integration map

The system should isolate broker-specific behavior behind a thin adapter. This makes the rest of the stack easier to test and protects the analytics layer from session-management details. The adapter should expose methods such as `connect`, `disconnect`, `resolve_contract`, `request_option_chain`, `subscribe_market_data`, `cancel_market_data`, and `request_historicalBars`. It should transform broker callbacks into normalized internal events.

Need	IBKR-facing capability	Normalized output	Notes
Session management	<code>connect</code> / <code>heartbeat</code> / <code>reconnect</code>	<code>session_state</code> events	Own backoff and retry behavior in one place
Universe discovery	option-chain resolution and contract lookup	canonical instrument rows	Persist raw payloads for audit
Live market data	underlying and option quote subscriptions	raw quote events	Keep callbacks lightweight
Historical retrieval	bars or snapshots for replay/bootstrap	historical raw partitions	Use same schemas as live where possible
Broker computations	option computation fields if available	diagnostic quote features	Never rely exclusively on broker analytics

Practical adapter pattern

Implement an adapter class that owns the broker connection and converts all callbacks into internal events. The rest of the codebase should never import broker callback enums directly. It should consume a broker-agnostic event stream. This pattern pays off immediately when writing replay tests because the replay source can emit the same internal event objects as the live adapter.

C. Data schemas and table contracts

The platform should maintain a small number of carefully designed tables rather than a sprawl of one-off datasets. The tables below are the minimum recommended schema families.

Table family	Purpose	Required primary keys
instrument_master	Canonical underlyings and option contracts	instrument_key, as_of_date
raw_market_events	Immutable tick/field observations	session_id, event_id
market_state_snapshots	Time-aligned inputs to analytics	snapshot_ts, instrument_key
forward_curve	Forward and implied carry diagnostics	snapshot_ts, underlying, maturity
iv_points	Solved implied-volatility observations	snapshot_ts, contract_key
surface_parameters	Model parameters by slice or global fit	snapshot_ts, underlying, maturity, model_version
surface_grid	Regularized grid values for use by other services	snapshot_ts, underlying, maturity, moneyness_bucket
pricing_results	Model price and Greeks by contract	snapshot_ts, contract_key, pricer_version
positions	Source-of-record positions or hypothetical positions	valuation_ts, portfolio_id, contract_key
risk_aggregates	Grouped risk outputs	valuation_ts, portfolio_id, group_key
scenario_results	Stress PnL outputs	valuation_ts, portfolio_id, scenario_id, contract_key
qc_results	Validation outcomes and diagnostics	run_id, check_name, target_key

Schema design rules

- Never store decimals as formatted strings. Use numeric types and explicit scales.
- Keep version fields on all derived analytics tables.
- Store timezone-aware timestamps or store UTC plus a clear convention; do not mix local and UTC ambiguously.
- Every derived table should reference the source snapshot_ts used to compute it.
- QC results must point to both the failing object and the run that produced the failure.

D. Detailed quality-control checklist

The QC layer should be formalized as a library of named checks. Each check returns a status, a severity, the measured value, the threshold version applied, and a payload of context fields. The checks below should exist at minimum in the first production release.

- Collector continuity: no unexplained gap longer than N seconds during liquid session.
- Underlying quote health: spread percentage and staleness below configured thresholds.
- Option chain coverage: minimum count of eligible calls and puts per monitored maturity.
- Forward stability: weighted forward estimate within tolerance of the median candidate.
- Parity residual: per-strike residuals below threshold for the accepted set.
- IV solver convergence: convergence ratio above threshold and residual distribution acceptable.
- Surface fit error: root-mean-square fit error within threshold by maturity.
- Calendar sanity: total variance should not move backwards across neighboring maturities without a fail flag.
- Greek sanity: finite-difference and analytic Greeks agree within tolerance on test contracts.
- Scenario completeness: all configured scenarios executed and stored with no missing results.

Robust outlier metrics

For several QC checks, robust statistics are preferable to means and standard deviations. For example, forward candidates and preliminary IV points often contain a few bad quotes. Median-based statistics resist contamination better and are easier to explain in ops reviews.

$$Z_i = \frac{x_i - \text{median}(x)}{1.4826 \text{MAD}(x)}$$

Equation 24. Robust z-score using median absolute deviation

E. Testing strategy

The platform requires four layers of tests. First, unit tests for pure math and data-transformation functions. Second, integration tests for the broker adapter using mocks or recorded event streams. Third, regression tests using archived market-state snapshots and expected outputs. Fourth, operational tests that verify scheduler entry points, logging, and basic health-check behavior.

- Unit tests: pricing identities, inversion convergence, parity calculations, date-count logic, and scenario aggregation.
- Integration tests: event normalization, contract resolution, quote subscription setup, reconnect handling.
- Regression tests: known chains with frozen expected forward curves, IV points, and surface diagnostics.

- Operational tests: CLI jobs run, partitions written, QC reports generated, alerts fired on simulated failure.

Reference test cases

Create a small immutable fixture library. It should contain a few liquid chains, a few pathological chains, and a few synthetic cases whose correct answers are analytically obvious. These fixtures are more valuable than a large number of shallow tests because they allow every release to be judged against known good and known bad conditions.

F. End-to-end processing sequence

The sequence below is the recommended canonical run order for one business day. It applies both to live production and to replay, with the only difference being the source of events. In production, the source is the broker adapter. In replay, the source is the stored raw-event partition.

1. Start connectivity service and verify session health.
2. Refresh or validate instrument master for the session date.
3. Launch underlying and option collectors; begin writing immutable raw events.
4. Create normalized market-state snapshots on the configured cadence.
5. Build forward curves by maturity from the latest accepted snapshot.
6. Solve implied volatilities for the filtered option set.
7. Fit and publish surface slices and the cross-maturity grid.
8. Compute model prices and Greeks for all eligible contracts.
9. Join positions and calculate line-level and aggregate risk.
10. Run scenario engine and publish stress summaries.
11. Run QC suite and generate operator dashboard/report.
12. Archive artifacts, close the day, and prepare replay partitions.

G. Pseudocode for key services

Collector service pseudocode

```
class RawCollector:
    def __init__(self, broker_adapter, writer, clock, config):
        self.adapter = broker_adapter
        self.writer = writer
        self.clock = clock
        self.config = config

    def on_event(self, broker_event):
        normalized = normalize_event(broker_event, receipt_ts=self.clock.now_utc())
        self.writer.append("raw_market_events", normalized)
```

```
def run(self):
    self.adapter.connect()
    self.adapter.subscribe_underlyings(self.config.underlyings)
    self.adapter.subscribe_options(self.config.option_universe)
    self.adapter.set_callback(self.on_event)
    self.adapter.heartbeat_loop()
```

Snapshot builder pseudocode

```
def build_snapshot(events, snapshot_ts, config):
    state = latest_by_field_before(events, snapshot_ts)
    underlying = build_underlying_state(state, config)
    option_rows = build_option_rows(state, snapshot_ts, config)
    return MarketStateSnapshot(
        snapshot_ts=snapshot_ts,
        underlying_state=underlying,
        option_rows=option_rows,
        flags=derive_state_flags(underlying, option_rows, config),
    )
```

Forward engine pseudocode

```
def estimate_forward(snapshot, maturity, rate, config):
    candidates = []
    for strike in eligible_strikes(snapshot, maturity, config):
        call_mid = mid(snapshot.call_quote(strike, maturity))
        put_mid = mid(snapshot.put_quote(strike, maturity))
        if is_usable(call_mid, put_mid):
            f_i = strike + math.exp(rate * maturity.t) * (call_mid - put_mid)
            weight = liquidity_weight(snapshot, strike, maturity, config)
            candidates.append((strike, f_i, weight))
    cleaned = reject_outliers(candidates, method="mad", config=config)
    return weighted_average(cleaned)
```

Surface engine pseudocode

```
def fit_surface(iv_points, config):
    slices = []
    for maturity, pts in group_by_maturity(iv_points):
        x = [p.log_moneyness for p in pts]
        y = [p.total_variance for p in pts]
        fit = fit_svi_or_fallback(x, y, config)
        slices.append(fit)
    return interpolate_across_maturities(slices, config)
```

H. Failure modes and responses

A junior operator must know what to do when the system deviates from nominal behavior. The table below intentionally translates technical issues into operational actions.

Failure mode	Likely cause	Immediate response	Longer-term fix
--------------	--------------	--------------------	-----------------

Industrial Roadmap for a Volatility Infrastructure Platform

Collector reconnect loop	Gateway session expired or network instability	Check session process, restart adapter, confirm client ID not duplicated	Harden heartbeat, improve process supervision
Sudden drop in eligible quotes	Entitlement issue, market closure, or stale quotes	Inspect collector summary and exchange status	Add exchange-calendar gating and stale-data diagnostics
Forward curve unstable at one maturity	Poor call-put parity from illiquid strikes	Inspect candidate strikes and QC reason codes	Tighten strike selection or outlier thresholds
Surface fit failure	Too few accepted points or pathological quote set	Publish fail flag and retain raw points for review	Improve fallback interpolation path
Greeks disagree with benchmarks	Unit convention drift or pricer regression	Run regression suite, freeze deployment	Strengthen test coverage and interface contracts
Scenario results missing	Upstream analytics incomplete or orchestration failure	Check dependency graph and rerun missing partitions	Add dependency-aware scheduling and alerting

I. Deployment model

The first institutional-grade deployment does not require exotic infrastructure. A modest but disciplined architecture is sufficient: one always-on collector process, one scheduler, one metadata database, one object or file store, and one monitoring stack. The key is process isolation. Keep the broker connectivity service separate from compute-heavy analytics jobs. This minimizes the chance that a CPU spike in analytics causes dropped market data.

- Development: local workstation, IBKR paper or non-production session, local file store, local scheduler.
- Staging: dedicated VM or small server, production-like configs, replay from stored data, full QC and dashboards.
- Production: dedicated collector host, separate analytics host if possible, supervised services, alerting, immutable artifact storage.

J. Release management

Every change that can alter economics must have a release artifact. That includes solver tolerances, QC thresholds, scenario grids, and smoothing settings. The release should specify what changed, why it changed, what tests passed, and which historical periods were revalidated. Never deploy a mathematically meaningful change by editing a notebook and rerunning the daily job.

- Use semantic versioning or date-based release tags for the analytics package.
- Keep configuration versions separate from code versions.
- Require regression results before promoting a release.
- Log the exact package version and config hashes into every derived table.

Part V — Expanded mathematical notes and engineering guidance

1. Forward-engine design details

The forward-engine implementation should be treated as a first-class analytics product. A common mistake is to compute a single parity forward from the apparent ATM strike and move on. That is rarely robust enough for production. Instead, the engine should start by defining an eligibility band around the estimated money region. Within that band, it should build candidate parity forwards per strike, compute liquidity weights, remove outliers, and summarize residuals. The final chosen forward should be accompanied by a confidence score derived from the density and consistency of the candidate set.

The confidence score is useful operationally because not every maturity is equally informative. For example, a maturity with six liquid strikes and tight spreads should produce a high-confidence forward. A maturity with two sparse strikes and wide spreads should produce a low-confidence forward and potentially trigger a fallback policy. The fallback policy should be explicit: either interpolate from neighboring maturities, borrow the previous trusted snapshot, or mark the maturity unusable. The policy chosen should depend on the use case, but the decision must be logged.

$$\text{Mid} = \frac{\text{Bid} + \text{Ask}}{2}, \quad \text{SpreadPct} = \frac{\text{Ask} - \text{Bid}}{\text{Mid}}$$

Equation 25. Mid-price and spread percentage diagnostics

Recommended forward diagnostics

- Candidate count used before and after outlier rejection.
- Median and weighted-mean parity forwards.
- Median absolute deviation of forward candidates.
- Residual of each candidate relative to the chosen forward.
- Forward confidence score and reason code if downgraded.

2. Solver engineering details

The inversion engine should separate economics from numerics. Economics determines the admissible price interval and the pricing function. Numerics determine how the root is found. This separation is valuable because it allows the same solver skeleton to be reused with different pricers. For a European option, the root function is the difference between the model price under

Black-style dynamics and the market price. For an American option, the same outer skeleton can call a lattice pricer instead.

Use bracketed methods wherever possible for reliability. Newton-type methods can be added as accelerators, but only inside a safe bracket. On illiquid or near-intrinsic quotes, pure Newton iterations can diverge or run into regions where the Vega collapses, making updates unstable. A junior implementer should prioritize a slightly slower but monotone-convergent approach over a faster but brittle solver.

- Lower volatility bound should be near zero but not exactly zero if the pricing function becomes numerically unstable there.
- Upper bound should be high enough to bracket distressed or event-rich markets but not so high that the function loses discrimination.
- Convergence should be measured both in price residual and parameter step size.
- Failed solves should return structured diagnostics, not NaN without context.

3. Surface-calibration guidance

A junior developer should think of calibration as a mapping problem: market points in, regularized surface out. The surface is useful only if the mapping is stable. Calibration stability depends at least as much on data hygiene as on the chosen model. Therefore, before tuning a parameterization, inspect the accepted points, quote density by maturity, and residual distribution. In many cases, improvements come from better QC rather than from a more complicated surface model.

When using SVI or any parametric family, store both the calibrated parameters and a reconstructed grid of total variance values. The parameter vector alone is not enough for operations. Operators and downstream services need a directly queryable surface grid, and they need diagnostics that compare the raw accepted points to the reconstructed values. The system should provide both. In sparse maturities, prefer a conservative fallback that is smooth and flagged over an aggressive calibration that produces sharp but unreliable local features.

- Calibrate per maturity slice first; only then handle cross-maturity interpolation.
- Fit in total variance space, not raw volatility, when possible.
- Apply sensible bounds to parameters and log bound hits.
- Keep a fallback nonparametric smoother for sparse slices.
- Always expose fit error metrics and accepted-point counts.

4. Greeks methodology

Greeks should be computed using one canonical methodology per product family. If analytic Greeks are available for the chosen pricing model, they should usually serve as the default. If finite-difference Greeks are used, the bump sizes must be versioned and documented. A common source of hidden error is inconsistent bump sizing across modules, which causes the risk engine and scenario engine to disagree for reasons unrelated to economics.

Finite-difference validation should still exist even when analytic Greeks are used. For a regression sample of contracts, compare analytic values with central-difference estimates under fixed bumps. The purpose is not to replace analytic formulas but to catch sign errors, unit mistakes, and accidental inconsistencies in the implementation.

- Define whether vega is per unit volatility or per one volatility point.
- Define whether theta is expressed per calendar day, trading day, or year fraction.
- Ensure monetized Greeks use the correct contract multiplier and currency.
- Store both raw and monetized sensitivities when useful.

5. Scenario methodology

Scenarios should be treated as explicit market states, not just as Greek multipliers. The best practice is to support both full repricing and local approximations. Full repricing is slower but more trustworthy and should be the source of truth for the daily scenario report. Local approximations are useful for exploratory intraday monitoring and sanity checks. The exact scenario grid should be version-controlled and retrievable alongside every scenario result.

A good default scenario family includes: parallel spot moves, parallel implied-volatility shifts, a combined spot-and-vol stress, and a small time roll-down for theta-style decay effects. Depending on the products covered, the grid may later add skew twists, forward shifts, or rate shocks. Those can be layered in without changing the architecture as long as the scenario object model is clean from the start.

Part VI — Operational runbooks

Runbook 1 — Start of day

Runbook 1 — Start of day should be written as a checklist that an operator can follow during a live session. Below is the minimum recommended content.

1. Verify the scheduler, collector host, and storage endpoints are healthy before market open.
2. Check that the IBKR session is authenticated and receiving heartbeats.
3. Confirm the instrument master for the day has been refreshed or explicitly waived.
4. Confirm that market-data entitlements are active for the monitored products.
5. Open the dashboard and verify zero backlog in the raw-event writer.
6. Run the bootstrap smoke test for one underlying and one option contract.
7. Document any known degraded mode before the session begins.

Runbook 2 — Intraday health monitoring

Runbook 2 — Intraday health monitoring should be written as a checklist that an operator can follow during a live session. Below is the minimum recommended content.

1. Watch event-rate dashboards for both underlyings and options.

2. Monitor the stale-quote ratio and collector reconnect count.
3. Inspect the latest forward-confidence score distribution.
4. Review solver convergence ratio and fit-error metrics for the active underlyings.
5. Check that the latest scenario report completed on schedule.
6. Escalate if any QC fail persists beyond the configured tolerance window.

Runbook 3 — End of day

Runbook 3 — End of day should be written as a checklist that an operator can follow during a live session. Below is the minimum recommended content.

1. Ensure all collector sessions closed cleanly and raw partitions are finalized.
2. Run end-of-day snapshot reconciliation and surface rebuild if required.
3. Publish final risk and scenario reports for the day.
4. Generate and archive the QC report and triage table.
5. Record run identifiers, code version, config hashes, and operator notes.
6. Back up or replicate the day's artifacts according to retention policy.

Runbook 4 — Replay / backfill

Runbook 4 — Replay / backfill should be written as a checklist that an operator can follow during a live session. Below is the minimum recommended content.

1. Select the date range and verify raw partitions exist for every date.
2. Pin the code version and configuration version used for replay.
3. Run snapshot rebuild, forward build, IV solve, surface build, risk, and QC in sequence.
4. Write outputs to versioned historical partitions rather than overwriting prior analytics.
5. Compare replay diagnostics with any live outputs available for the same dates.
6. Archive the replay manifest and summary statistics.

Runbook 5 — Incident response

Runbook 5 — Incident response should be written as a checklist that an operator can follow during a live session. Below is the minimum recommended content.

1. Acknowledge the alert and identify whether the incident is in connectivity, data, analytics, or orchestration.
2. Check the latest healthy heartbeat and the last successful write to each critical table.
3. Collect logs around the failure window using the run and correlation identifiers.
4. Perform the documented restart procedure for the affected service only.

5. If data loss is suspected, quarantine downstream jobs until replay or gap-filling can be performed.
6. Write a short incident note capturing impact, root cause, remediation, and follow-up actions.

Operator escalation policy

Not every QC warning requires immediate intervention. The escalation policy should distinguish between informational warnings, action-required warnings, and hard failures. For example, a small rise in fit error may warrant observation, while a collector outage or a missing scenario report is a hard failure requiring intervention. Define severity levels up front and map each to an expected response time and owner.

- Severity 1: data collection halted or storage unavailable. Immediate intervention required.
- Severity 2: analytics incomplete or QC hard failure on a monitored underlying. Same-session intervention required.
- Severity 3: quality degradation within tolerances but trending worse. Investigate and track.
- Severity 4: informational events and low-risk anomalies. Log and review later.

Part VII — Configuration design and example artifacts

Configuration philosophy

Configurations are economic inputs. They should never live as scattered constants in notebooks or inside implementation files. Every threshold, bump size, scenario grid, strike-selection rule, and cadence should be represented in a versioned configuration artifact. The configuration package should support inheritance so that a base institutional configuration can be specialized by environment or product family without duplicating the entire tree.

Suggested configuration files

- `environment.yaml` — storage paths, service endpoints, log levels, scheduler settings.
- `broker.yaml` — client IDs, reconnect policy, session windows, market-data cadence.
- `universe.yaml` — monitored underlyings, exchanges, product families, maturity windows.
- `qc.yaml` — quote filters, stale limits, solver thresholds, fit tolerances.
- `scenarios.yaml` — named stress scenarios, shifts, combinations, and report subsets.
- `pricing.yaml` — solver bounds, finite-difference bumps, pricer choices by product family.

Illustrative configuration snippet

```
# qc.yaml
quote_filters:
  max_spread_pct: 0.25
  max_quote_age_seconds: 60
  min_open_interest: 10
  require_positive_bid: true
```

```
forward_engine:
  strike_band_mode: nearest_liquid
  max_candidate_count: 12
  outlier_method: mad
  max_robust_zscore: 3.5
```

```
iv_solver:
  lower_vol: 0.0001
  upper_vol: 5.0000
  price_tolerance: 1.0e-6
  max_iterations: 100
```

```
surface:
  model: svi
  fallback_model: spline
  min_points_per_slice: 5
  max_rmse: 0.02
```

Part VIII — Acceptance tests by module

Connectivity and access

1. A fresh environment can establish a session and retrieve at least one quote.
2. Heartbeats are observable and reconnects are logged with timestamps.
3. A wrong credential or entitlement failure produces a clear, structured error.

Universe service

1. The active option chain can be reproduced for a fixed as-of date.
2. Duplicate contract records are removed deterministically.
3. Multiplier, currency, expiry, and exchange are populated for every active contract.

Collector service

1. Raw events are written continuously during market hours.
2. Process restart does not corrupt or duplicate records beyond documented idempotency behavior.
3. Session summary contains event counts and reconnect counts.

Forward engine

1. Weighted forward estimate is stable under small strike-set perturbations.
2. Outlier candidates are rejected for the documented reason.
3. Diagnostics explain every poor-quality maturity.

IV solver

1. Reference contracts converge to expected IVs.
2. Out-of-bounds prices return structured solver failures.
3. Residual and iteration count are stored per solve.

Surface engine

1. Accepted points and fitted values are queryable together.
2. Surface fit parameters are reproducible given the same inputs.
3. Calendar monotonicity diagnostic is computed for all monitored maturities.

Risk and scenarios

1. Portfolio aggregates reconcile to line-level sums.
2. Scenario outputs are reproducible from snapshot, positions, and scenario version.
3. Top contributors can be identified for every worst-case scenario.

Part IX — Data dictionary

instrument_key: Canonical unique key for a tradable contract or underlying.

contract_id_broker: Broker-specific contract identifier as returned by IBKR.

snapshot_ts: Timestamp of the normalized market-state snapshot used by analytics.

exchange_ts: Timestamp associated with the originating market observation when available.

receipt_ts: Timestamp when the event reached the collector.

reference_spot: Chosen underlying reference price used for downstream analytics.

reference_type: Label describing whether reference_spot came from mid, last, close, or fallback.

maturity_years: Year fraction between valuation time and expiry under the chosen day-count convention.

forward_price: Chosen forward estimate for a specific maturity.

implied_carry: Carry or dividend-like quantity implied by spot and forward.

log_moneyness: $\ln(\text{strike} / \text{forward_price})$.

mid_option_price: $(\text{bid} + \text{ask}) / 2$ for the accepted option quote.

implied_vol: Volatility solved from price and model assumptions.

total_variance: implied_vol^2 multiplied by maturity_years.

fit_quality_flag: Indicator describing calibration or QC status.

delta: First derivative of price with respect to underlying reference spot.

gamma: Second derivative of price with respect to underlying reference spot.

vega: Derivative of price with respect to volatility under the chosen unit convention.

theta: Derivative of price with respect to time under the chosen convention.

dollar_gamma: Gamma monetized by spot squared and multiplier.

scenario_id: Versioned identifier of a defined stress scenario.

scenario_pnl: Portfolio or line-level revaluation under scenario_id.

qc_status: Pass/warn/fail classification for a named validation check.

Part X — Glossary

Canonical key: The internally defined identifier that uniquely names an instrument independently of any one vendor or broker session.

Carry: A generic term covering financing and distribution effects that cause forward to differ from spot.

Confidence score: A quality label or numeric score indicating how trustworthy a derived estimate is.

Derived analytics: Any object computed from raw observations, such as forwards, IV points, surfaces, Greeks, or scenarios.

Immutable raw layer: The append-only storage layer containing normalized observations exactly as captured or reconstructed.

Market-state snapshot: A time-aligned, deterministic view of underlying and option quotes used by analytics modules.

No-arbitrage diagnostic: A check designed to flag surface shapes that are economically implausible or inconsistent.

Replay: The process of recomputing analytics historically from stored raw data or snapshots using the same code path as live processing.

Scenario version: A named, version-controlled set of stress definitions used to compute scenario PnL.

Surface grid: A regularized table of volatility or total-variance values sampled from the fitted surface representation.

Part XI — Junior handover checklist

The final section is intentionally practical. A junior implementer or operator should be able to check off each item and demonstrate competence without needing any hidden context about the strategy that will eventually consume the analytics.

1. Can explain the difference between raw events, snapshots, and derived analytics.
2. Can run the connectivity smoke test and interpret failures.

IBKR-based volatility infrastructure blueprint

3. Can refresh the instrument master and inspect a discovered option chain.
4. Can locate the latest raw partition and the latest surface partition for a chosen underlying.
5. Can explain how the forward for one maturity was chosen and where the diagnostics are stored.
6. Can rerun the IV solver for one contract from a saved snapshot.
7. Can plot accepted IV points and fitted surface slice for one maturity.
8. Can interpret line-level Greeks and aggregate risk tables.
9. Can run the scenario report and identify the worst-case scenario and contributors.
10. Can read the QC report and navigate to the triage table for a failed maturity.
11. Can restart a failed collector without corrupting the raw store.
12. Can launch a historical replay for a chosen date range and compare outputs with prior versions.

Part XII — File-by-file implementation guide

`connectivity/session.py`

Responsibility. Own connection lifecycle to TWS or IB Gateway. Expose `connect()`, `disconnect()`, `is_healthy()`, and `heartbeat_loop()`. Maintain a small state machine: `DISCONNECTED`, `CONNECTING`, `CONNECTED`, `DEGRADED`, `RECONNECTING`. The module should not know anything about volatility mathematics. Its only job is to provide a reliable transport for broker interactions.

- Implement exponential backoff with jitter for reconnects.
- Emit structured logs on every state transition.
- Prevent duplicate client IDs by configuration validation at startup.
- Expose a heartbeat timestamp and age metric.
- Refuse to claim health until at least one successful broker round-trip has completed.

`universe/contracts.py`

Responsibility. Resolve and normalize instruments. This file should provide the canonical dataclasses for underlyings and option contracts, plus serializer helpers. It should also define how instrument keys are constructed and parsed.

- Treat expiry, strike, right, exchange, currency, multiplier, and broker contract ID as separate fields.
- Add a method that round-trips from key string back into object form.
- Keep raw broker payloads in an adjacent storage layer for audit.
- Validate strike type conversion carefully to avoid integer/float drift.

- Add explicit tests for duplicate chain entries and missing multipliers.

collectors/raw_writer.py

Responsibility. Persist normalized events quickly and safely. This module should not perform economics or analytics. It should batch writes where appropriate, flush on controlled intervals, and tag every write with collector session metadata.

- Use append-only semantics.
- Validate the schema before write, not after.
- Handle disk or object-store failures by surfacing a hard alert.
- Add per-partition write counters for operational reporting.
- Never silently drop malformed events; quarantine them with a reason code.

snapshots/builder.py

Responsibility. Convert raw events into deterministic market-state snapshots. This module should define field precedence, quote-age limits, fallback logic, and the exact assembly of snapshot objects.

- Write pure functions for `latest_by_field`, `choose_reference_spot`, and `build_option_row`.
- Include state flags in the output object rather than relying on logs alone.
- Keep the builder deterministic by sorting events explicitly.
- Version the cadence and staleness thresholds via configuration.
- Provide a debug mode that prints the exact raw events chosen for one contract.

forwards/engine.py

Responsibility. Build the forward curve and implied carry diagnostics. This file should contain candidate generation, weighting, outlier rejection, smoothing, and confidence-scoring logic.

- Separate candidate construction from candidate selection.
- Keep every intermediate candidate row for audit.
- Add a minimum candidate-count threshold before accepting a forward.
- Expose both weighted mean and median diagnostics.
- Include a fallback policy object rather than hard-coding the fallback.

iv/solver.py

Responsibility. Price-to-vol inversion. The module should expose scalar and vectorized solve functions plus structured diagnostics objects. All convergence decisions should be explicit and tested.

- Use a bracketed solver as the primary safe path.
- Return diagnostics objects even on success.

Industrial Roadmap for a Volatility Infrastructure Platform

- Document exactly which pricing model is assumed for each product family.
- Implement intrinsic and upper-bound checks before entering the root finder.
- Make the scalar path maximally readable and cover it with tests.

surfaces/calibration.py

Responsibility. Fit surface slices and produce a regularized grid. The module should not fetch data itself. It should consume accepted IV points and emit parameters plus diagnostics.

- Persist raw points, accepted points, rejected points, fit parameters, and reconstructed grid values.
- Bound parameters and log bound hits.
- Compute RMSE and max error by slice.
- Support at least one fallback method for sparse slices.
- Add plotting helpers for operations and regression review.

pricing/european.py and pricing/american.py

Responsibility. Canonical pricing functions and Greeks. These files form the trusted pricing library for the platform. All downstream analytics should call these files rather than reimplementing formulas elsewhere.

- Expose typed inputs and outputs.
- Keep unit conventions explicit in docstrings.
- Provide benchmark fixtures covering limiting cases.
- Separate pure pricing from vectorization wrappers.
- Test consistency between price and numerical derivatives.

risk/aggregation.py

Responsibility. Merge positions with analytics results and produce line-level and aggregate sensitivities. The module should support grouping by any configured key such as underlying, maturity bucket, or desk category.

- Preserve line-level outputs for audit.
- Add currency handling if multiple currencies are present.
- Keep aggregate formulas transparent and queryable.
- Reconcile to broker Greeks only as diagnostics, never as the source of truth.
- Version the risk snapshot with analytics version and position source timestamp.

risk/scenarios.py

Responsibility. Reprice portfolios under named scenarios. The module should support both full repricing and local approximations and expose contributor analysis.

- Define a typed Scenario object.

- Store every scenario with version and parameters.
- Allow scenario families to be filtered by report type.
- Separate scenario generation from scenario execution.
- Include top-contributor extraction in the core API.

qc/checks.py

Responsibility. Implement validation checks as named reusable functions. Each check returns a measured value, a status, and supporting context.

- Keep thresholds in config, not in code.
- Assign stable reason codes to every fail path.
- Write summary and detail outputs separately.
- Track historical distributions of QC measures for trend analysis.
- Document operator meaning for each check in the module README.

orchestration/jobs.py

Responsibility. Provide entry points for scheduled jobs. Jobs should be thin wrappers that orchestrate library calls, write manifests, and emit metrics.

- Use dependency ordering so downstream jobs do not run on incomplete upstream data.
- Write one manifest per job with parameters, versions, and outputs.
- Ensure reruns are idempotent or explicitly versioned.
- Surface clear exit codes for the scheduler.
- Add dry-run modes for operational safety.

Part XIII — Extended appendices

Appendix A — Detailed formulas and implementation comments

Reference spot

The chosen spot should be the result of a deterministic selection rule applied to underlying quotes. Store the chosen reference plus a type flag and all candidate fields used to choose it.

Forward parity

The parity forward is an empirical estimate from market quotes. Because market quotes are noisy, store strike-level forward candidates before smoothing or weighting.

Carry identity

The spot-forward-carry identity is valuable both for diagnostics and for plugging into product-specific pricers. The developer should document the day-count basis used for T.

Total variance

Surface interpolation should prefer total variance because many cross-maturity pathologies become easier to see in variance space than in raw volatility space.

Greek PnL approximation

The local approximation is a diagnostic convenience, not a replacement for full repricing under large shocks. The scenario engine should therefore support both.

$$S_t^{\text{mid}} = \frac{B_t + A_t}{2}$$

$$F(T) \approx K + e^{rT}(C(K, T) - P(K, T))$$

$$F(T) = S_0 e^{(r - q)T}$$

$$q(T) = r(T) - \frac{1}{T} \ln\left(\frac{F(T)}{S_0}\right)$$

$$k = \ln\left(\frac{K}{F(T)}\right)$$

$$w(k, T) = \sigma_{\text{imp}}(k, T)^2 T$$

$$d_1 = \frac{\ln(S_0/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T}$$

$$C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2)$$

$$P = K e^{-rT} N(-d_2) - S_0 e^{-qT} N(-d_1)$$

$$V_{n,j} = \max(\Phi(S_{n,j}), e^{-r\Delta t}[pV_{n+1,j+1} + (1-p)V_{n+1,j}])$$

$$\Delta = \frac{\partial V}{\partial S}$$

$$\Gamma = \frac{\partial^2 V}{\partial S^2}$$

$$\mathcal{V} = \frac{\partial V}{\partial \sigma}$$

$$\Theta = \frac{\partial V}{\partial t}$$

$$\text{DollarGamma} = \Gamma S^2 \times \text{Multiplier}$$

$$\text{DollarVega} = \mathcal{V} \times \text{Multiplier}$$

$$\Delta V \approx \Delta dS + \frac{1}{2} \Gamma (dS)^2 + \mathcal{V} d\sigma + \Theta dt$$

$$w(k) = a + b \left(\rho(k - m) + \sqrt{(k - m)^2 + \sigma^2} \right)$$

$$\frac{\partial w(k, T)}{\partial T} \geq 0$$

$$\sigma_I^2 \approx \sum_i w_i^2 \sigma_i^2 + \sum_{i \neq j} w_i w_j \rho_{ij} \sigma_i \sigma_j$$

Appendix B — Sample daily manifest

```
{
  "run_id": "2026-04-06_eod_001",
  "environment": "production",
  "code_version": "vol-infra-4.0.0",
  "config_hashes": {
    "universe": "u_8d6...",
    "qc": "q_34b...",
    "pricing": "p_c21...",
    "scenarios": "s_55a..."
  },
  "input_partitions": {
    "raw_market_events": "dt=2026-04-06",
    "positions": "ts=2026-04-06T21:00:00Z"
```

IBKR-based volatility infrastructure blueprint

```
    },  
    "output_partitions": {  
      "market_state_snapshots": "dt=2026-04-06",  
      "forward_curve": "dt=2026-04-06",  
      "iv_points": "dt=2026-04-06",  
      "surface_parameters": "dt=2026-04-06",  
      "risk_aggregates": "dt=2026-04-06",  
      "scenario_results": "dt=2026-04-06",  
      "qc_results": "dt=2026-04-06"  
    },  
    "status": "success"  
  }  
}
```

Appendix C — Suggested documentation pack

- `architecture_overview.pdf` — system diagram, data flow, service boundaries, and environment map.
- `environment.md` — exact setup steps, secrets model, bootstrap smoke test.
- `operating_runbooks.md` — start-of-day, intraday, end-of-day, replay, incident response.
- `schemas.md` — all table definitions with field descriptions and partitioning rules.
- `module_READMEs/` — one README per major package explaining public APIs and failure modes.
- `release_checklist.md` — regression expectations, approvals, rollback plan.
- `known_limitations.md` — current compromises, unresolved issues, planned enhancements.

Appendix D — Review questions for a junior engineer

1. Explain why the forward curve must be built before log-moneyness and surface fitting.
2. Explain the difference between a raw market event and a market-state snapshot.
3. Explain why a filtered quote set must preserve rejected-quote diagnostics.
4. Explain why deterministic replay is important for regression testing.
5. Explain why the platform stores both fitted surface parameters and reconstructed grid values.
6. Explain the difference between a QC warning and a hard QC failure.
7. Explain the difference between broker-returned Greeks and platform-native Greeks.
8. Explain why scenario definitions must be version-controlled.

Part XIV — Service-level objectives, monitoring, and operational metrics

An institutional-grade platform should define service-level objectives even if the first deployment is modest in scale. The point is to make expectations explicit. Without SLOs, the team cannot distinguish between acceptable degradation and unacceptable operational failure. The SLOs below are representative and should be tuned to the actual business context, but the habit of defining them should exist from the first release.

Industrial Roadmap for a Volatility Infrastructure Platform

Service	Key metric	Suggested target	Operator action if missed
Connectivity	Heartbeat age	< 30 seconds during session	Investigate gateway, session auth, network
Collector	Raw-event write lag	< 5 seconds median	Check writer backlog and storage endpoint
Snapshots	Latest snapshot freshness	< configured cadence + tolerance	Inspect snapshot job and raw-event completeness
Forward engine	Maturity coverage ratio	> 95% of monitored maturities	Inspect quote QC and parity diagnostics
IV solver	Convergence ratio	> 97% on accepted quotes	Review solver residuals and quote quality
Surface engine	Completed monitored surfaces	> 95% on monitored underlyings	Inspect sparse maturities and fallback path
Scenario engine	Report completion before cut-off	100%	Check dependency graph and runtime bottlenecks
QC framework	QC report publication	100% daily	Investigate failed checks or orchestration issues

Recommended metrics catalog

- Collector metrics: raw events per second, reconnect count, heartbeat age, write lag, malformed-event count.
- Snapshot metrics: snapshots produced, stale-underlying ratio, stale-option ratio, fallback-spot ratio.
- Forward metrics: candidate count per maturity, confidence score distribution, outlier rejection rate.
- IV metrics: solve count, convergence ratio, median iterations, residual distribution, bounds-hit count.
- Surface metrics: accepted-point count, fit RMSE distribution, fallback-fit frequency, calendar-check failures.
- Risk metrics: aggregate delta, gamma, vega, theta by portfolio and by underlying, timestamp age.
- Scenario metrics: scenario runtime, result count, worst-case loss, top-contributor concentration.

- QC metrics: check pass/warn/fail counts, unresolved triage count, repeat-failure counts by underlying.

Dashboard design

The operator dashboard should have three layers. The first layer is system health: are services running, is data flowing, are tables being written. The second layer is analytics health: are forwards stable, are IVs converging, are surfaces fitting. The third layer is risk/report health: are the latest risk and scenario reports current and complete. Do not overload the front page with too many charts. The dashboard should answer the operational questions in under one minute.

Part XV — Data retention, lineage, and governance

Even a privately operated volatility platform benefits from governance discipline. The most important governance principle is lineage: every derived object should point back to the raw observations, code version, and configuration version that produced it. The second principle is retention: store enough history to support audit, debugging, and replay, but define clear retention tiers so storage costs and operational complexity remain controlled.

Retention tiers

- Tier 1 — Raw events: highest fidelity; long enough retention to support replay and forensic debugging.
- Tier 2 — Normalized snapshots: retained longer than raw if storage requires compromise, because they are compact and analytically rich.
- Tier 3 — Derived analytics: forwards, IV points, surfaces, risk, and scenarios; retained for long-horizon trend analysis and audit.
- Tier 4 — Summary reports and manifests: retained indefinitely or near-indefinitely because they are small and operationally valuable.

Lineage requirements

- Every derived record must include source snapshot_ts and source data partition identifiers.
- Every job must emit a manifest with code version and config hashes.
- Any replay or backfill must write a new version identifier instead of silently mutating past results.
- QC outputs should reference both the failing object and the run that generated the object.

Change-control categories

Not all changes carry the same risk. The team should classify changes into at least three categories. Category A changes alter economics directly, such as pricer logic, solver bounds, surface parameterization, or scenario definitions. Category B changes alter validation or operational behavior, such as thresholds, alert policies, or run schedules. Category C changes are non-economic, such as log-message improvements or documentation fixes. Category A changes should require the strongest regression evidence and the most explicit sign-off.

Part XVI — Extended test matrix

A production handover is much stronger when the tests are presented as a matrix rather than as a loose list. The matrix below is not exhaustive, but it shows how to think about coverage across module type, data condition, and expected outcome.

Module	Condition	Test input	Expected result	Priority
Spot builder	Normal market	Tight underlying bid/ask	Reference type = mid; spread diagnostics valid	High
Spot builder	Wide market	Bid/ask spread beyond threshold	Fallback triggered and labeled	High
Forward engine	Liquid maturity	Multiple near-money call/put pairs	Stable weighted forward with high confidence	High
Forward engine	Sparse maturity	Two poor-quality pairs only	Low-confidence or rejected maturity	High
IV solver	Normal quote	Tradable mid in no-arbitrage bounds	Converged implied vol with diagnostics	High
IV solver	Bad quote	Price below intrinsic value	Structured failure, no silent NaN	High
Surface engine	Dense slice	Many accepted points	Stable fit with low RMSE	High
Surface engine	Sparse slice	Few accepted points	Fallback fit or fail flag per policy	High
Pricing engine	Degenerate American case	No early-exercise value expected	American and European prices close	Medium
Risk aggregation	Multiple positions	Known synthetic book	Aggregates reconcile to line items	High
Scenario engine	Configured grid	Frozen snapshot and positions	Scenario count complete and deterministic	High

Replay pipeline	Historical day	Stored raw partitions	Replay equals same-code live logic	High
-----------------	----------------	-----------------------	------------------------------------	------

Regression-candidate dataset design

Create a curated library of replay days. The library should include calm days, event-heavy days, days with sparse liquidity, and days containing known operational incidents such as brief disconnects. Each replay day becomes a standard challenge set for new releases. This is one of the highest-leverage investments the team can make because it prevents regressions from hiding behind good behavior on only one benign day.

Part XVII — Extended coding examples

Dataclass pattern for analytics objects

```
from dataclasses import dataclass
from typing import Optional, List

@dataclass(frozen=True)
class ForwardCandidate:
    strike: float
    maturity_years: float
    call_mid: float
    put_mid: float
    forward_estimate: float
    weight: float
    quality_flag: str

@dataclass(frozen=True)
class ForwardResult:
    underlying: str
    snapshot_ts: str
    maturity_years: float
    chosen_forward: float
    confidence_score: float
    candidates: List[ForwardCandidate]
    diagnostics_version: str
```

Result-object pattern for solver diagnostics

```
@dataclass(frozen=True)
class IvSolveResult:
    contract_key: str
    snapshot_ts: str
    market_price: float
    implied_vol: Optional[float]
    converged: bool
    iterations: int
    residual: float
    lower_bound: float
```

```
upper_bound: float
failure_reason: Optional[str]
model_name: str
model_version: str
```

Scenario object pattern

```
@dataclass(frozen=True)
class Scenario:
    scenario_id: str
    spot_shift_pct: float
    vol_shift_abs: float
    time_roll_days: int
    description: str
    version: str
```

CLI entry point pattern

```
def main():
    args = parse_args()
    cfg = load_config(args.config_path)
    run = JobRunContext.from_args(args, cfg)
    logger = build_logger(run)

    snapshots = load_snapshots(run)
    forwards = build_forwards(snapshots, cfg, logger)
    iv_points = solve_chain_iv(snapshots, forwards, cfg, logger)
    surfaces = build_surfaces(iv_points, cfg, logger)
    write_outputs(run, forwards, iv_points, surfaces)
    publish_manifest(run)
```

Part XVIII — Frequently misunderstood implementation points

Why not compute analytics directly inside the broker callback?

Because callbacks should be short and loss-tolerant. Heavy analytics inside callbacks increase the probability of dropped events and make historical replay impossible to align with live logic.

Why keep rejected quotes?

Because rejected quotes are evidence. QC threshold tuning and operational debugging depend on understanding what was excluded and why.

Why version configurations separately from code?

Because a threshold or scenario-grid change can alter economics without any code edit. Separate versioning preserves lineage.

Why store both fitted parameters and grid values?

Because parameters are concise but not directly convenient for every downstream consumer. Grid values are easy to inspect, plot, and join to risk code.

Why insist on replaying with the same code path as live?

Because dual paths drift. If historical and live analytics differ by implementation path, regressions become impossible to reason about.

Why treat forwards as first-class outputs rather than internal intermediates?

Because forward quality drives moneyness, IV, surface shape, and risk. Hiding forward diagnostics deprives the platform of one of its most informative health measures.

Part XIX — Final implementation reminders

The purpose of this final section is to prevent a junior engineer from taking common shortcuts that appear harmless but later make the platform unreliable. The project should be treated as a product with users, audits, and operational obligations. Every shortcut that erodes determinism or lineage is therefore costly, even if it appears to save time in the first week of implementation.

- Do not compute a forward, implied volatility, or Greek in an exploratory notebook and then copy the formula into production later. Production logic should be implemented in a tested library first and used by notebooks second.
- Do not overwrite historical analytics partitions without writing a new version identifier. Reproducibility requires visible history.
- Do not hide fallback behavior. If the system used last price instead of mid, or interpolated a missing maturity, that fact must be queryable.
- Do not allow the collector to perform expensive CPU-bound work. Separate ingestion from analytics.
- Do not merge strategy-specific assumptions into generic platform code. Keep the infrastructure reusable and opaque with respect to trading intent.
- Do not sign off on a release without replaying at least one calm day and one stressed day from the regression library.

Appendix E — Canonical field mapping checklist

The table below is intended as an implementation checklist during schema design and code review. Every row should exist somewhere in the codebase, with a single authoritative definition. This appendix exists because field ambiguity is one of the most persistent causes of downstream confusion in risk systems.

Field	Layer	Type	Implementation note
event_id	raw	string	Unique within

Industrial Roadmap for a Volatility Infrastructure Platform

			collector session; never recycled
collector_session_id	raw	string	Connects raw events to a collector run summary
instrument_key	all	string	Canonical key used across all tables
contract_id_broker	master/raw	string	Foreign key to broker-specific representation
field_name	raw	string	Name of observed field such as bid, ask, last, gamma
field_value	raw	numeric/string	Typed at normalization boundary
receipt_ts	raw	timestamp	UTC time when collector received the event
canonical_ts	snapshot	timestamp	Time used to align a snapshot
reference_spot	snapshot	numeric	Chosen spot used by downstream analytics
reference_type	snapshot	string	mid / last / close / fallback
quote_age_seconds	snapshot	numeric	Age of quote at snapshot time
mid_option_price	snapshot	numeric	Computed from bid and ask when eligible
forward_price	forward	numeric	Chosen forward per maturity
forward_confidence	forward	numeric	Quality score from candidate diagnostics
implied_vol	iv_points	numeric	Solved volatility
solver_converged	iv_points	bool	True only if convergence criteria

Industrial Roadmap for a Volatility Infrastructure Platform

			met
surface_model	surface	string	SVI, spline, or other configured method
fit_rmse	surface	numeric	Fit error for slice or surface
greek_delta	risk	numeric	Canonical delta
greek_gamma	risk	numeric	Canonical gamma
greek_vega	risk	numeric	Canonical vega
dollar_gamma	risk	numeric	Monetized gamma using multiplier
scenario_id	scenario	string	Versioned scenario key
scenario_pnl	scenario	numeric	Repriced PnL under scenario
qc_status	qc	string	pass / warn / fail
reason_code	qc	string	Stable diagnostic reason code